

A distributed framework for parallel data mining using HPJava

O F Rana and D Fisk

Java has become a language of choice for applications executing in heterogeneous environments utilising distributed objects and multithreading. To handle large data sets, scalable and efficient implementations of data mining approaches are required, generally employing computationally intensive algorithms. Conventional Java implementations do not directly provide support for the data structures often encountered in such algorithms, and they also lack repeatability in numerical precision across platforms.

This paper describes a distributed framework employing task and data parallelism, and implemented in high performance Java (HPJava). Issues of interest for data mining algorithms are identified, and possible solutions discussed for overcoming limitations in the Java Virtual Machine. The framework supports parallelism across workstation clusters, using the message passing interface as middleware, and can support different analysis algorithms, wrapped as Java objects, and linked to various databases using the Java database connectivity interface. Guidelines are provided for implementing parallel and distributed data mining on large data sets, and a proof-of-concept data mining application is analysed using a neural network.

1. Introduction

Organisations today are faced with making business decisions using large quantities of data, which may be maintained on different machines around the world. Such data may either be short-lived transactional data, or may be detailed, non-volatile data providing a consistent view of the organisation over a long period of time. To support decision making, various methods have been proposed to analyse patterns in such stored data, involving approaches from artificial intelligence, statistics, databases, optimisation theory and logic programming. The utility of such methods is generally determined by their speed of analysis, the range of business problems that can be successfully analysed, and the range of data sets that are supported. Such issues become particularly significant when dealing with large data sets, which are a first step in transforming a database system from a mechanism for reliable storage to one whose primary use is in decision support [1].

Associated with large data sets is the concept of on-line analytical processing (OLAP) which involves the interactive exploration of data, generally involving computationally intensive queries. Whereas traditional OLAP involved a human-driven exploration of data, current data mining approaches involve a computer-driven exploration, with data processing where the user does not know how to precisely describe the query. Examples of such problems are present in many business scenarios, from

analysing credit card usage data (to detect fraudulent activity), to looking for patterns in telecommunications data (to correlate faults with other regional events). Various data visualisation techniques are also needed to support interactive exploration of data, and an area of concern is the ability to deal with data sets with a high dimensionality.

Use of a distributed framework is suggested as this has the potential to make OLAP algorithms more widely available on desktop machines, rather than being restricted to execution on high-end machines. Subsequent increases in processing power of CPUs will also enhance our environment, as each component within the environment will have greater processing power. Use of commodity processors in the framework suggests that increases in the processing power of these processors, and improvements in network performance (such as reduced latency and higher data bandwidth) will further improve application performance. The proposed framework employs Java for linking together heterogeneous platforms running different operating systems. Various Java application programming interfaces (APIs) may be used to link databases capable of handling SQL statements, enable visualisation, and provide support for other devices that may be used to capture data in real time, such as smart cards and sensors. The Java model is extended to provide support for parallel computing, using the message passing interface (MPI) libraries [2], and ways in which parallel data mining may be implemented within

such an environment are described. The framework also provides integration of different services and data sources, allowing support for linking with legacy codes via the Java native interface (JNI).

The issues of relevance in parallel data mining, using commodity computing resources, are analysed, and, in particular, a framework which makes more efficient use of in-house computing resources is proposed, and the issues which arise from the development of such a framework are discussed. This paper is divided as follows — section 2 describes previous work in parallel data mining and OLAP, and how this current work differs from those earlier approaches, section 3 outlines the design aims behind high performance Java (HPJava) [3], and section 4 describes the framework for implementing parallel data mining. Section 5 contains a neural network implementation, and a discussion of the results obtained by varying the size of the data set and the number of workstations in a clustered computing environment. Section 6 outlines further work to be undertaken in this area.

2. Previous work

First generation data mining systems were focused on providing single or multiple analysis algorithms, whereas second generation systems aim to integrate data mining with data management systems, and provide support for predictive modelling and visualisation. Hence, current efforts are aimed at dealing with data which is distributed and highly heterogeneous, and provide mining capabilities when embedded or used as part of another system.

Albrecht and Lehner [4] provide an overview of different OLAP approaches in data warehouses with reference to the CUBESTAR system. They contrast an enterprise data warehouse providing integrated data from different business areas stored within a single corporate data model, with 'data marts' providing a high-focused version of a data warehouse, and developed to serve a small business group with well-defined needs. They identify areas of concern in integrating data marts to form a larger data warehouse, such as in the use and management of query aggregates, data allocation and load balancing via replication, and the fragmentation problem with reference to database management systems. Generally performance is preferred over accuracy, and achieved by changing the granularity of response — for instance, by reducing the consistency of results if only an overview is required.

Work in parallel data mining is distinguished by the particular method of analysis being employed, such as decision trees based on ID-3/C4.5 [5], neural networks [6], statistical methods [7], genetic algorithms [8] and others. SLIQ (Supervised Learning In Quest), developed by IBM's Quest project team, is a decision-tree classifier designed to handle large training data [9]. SLIQ pre-sorts attributes when building a decision tree, and maintains separate lists

for each continuous attribute and a memory-resident data structure called the 'class list'. Although SLIQ can handle disk-resident data efficiently, it still has to maintain the 'class list' in memory, which grows in direct proportion to the total number of records in the training set. Shafer et al identify classification as an important problem in data mining, and present a decision-tree-based parallel classification algorithm called SPRINT [10], which also comes from the IBM Quest team, but overcomes some of the restrictions in SLIQ.

Pendse [11] gives a good overview of commercial data mining and OLAP tools, dividing commercial tools according to two criteria — location of the processing and location of data storage. Hence, regardless of where the data processing takes place, there are three locations where data may be stored:

- in a relational database management system (RDBMS) (with no external storage),
- in a shared multidimensional database on a server,
- as local files on a client PC.

Similarly, processing engines may use multi-pass standard query language (SQL), use a server employing multi-user capability, or execute on the client PC. Several commercial products work in more than one way, generally making an exact classification difficult. Examples of RDBMS-based storage and multi-pass SQL products include the 'MicroStrategy DSS Agent', whereas tools employing RDBMS-based storage but other processing techniques include the IBM DB2 OLAP server, the Microsoft OLAP server, Oracle Express, etc. Some vendors produce desktop OLAP tools that can be integrated into other applications, such as Cognos PowerPlay, OEM-based tools from Business Objects, Brio Technology, MicroStrategy and AppSource.

Various software and hardware vendors also provide tools for parallel data mining, such as Torrent Systems, Tandem, Sun Microsystems and IBM [12], concentrating on either providing parallel data access from a particular database (DB2 UDB in the case of IBM), or running a particular application program in parallel. The emphasis is generally on particular hardware architectures, database systems or particular analysis packages such as SAS or Matlab. Parallel database access requires the vendor to provide a means of extracting and merging data obtained in parallel from a database, generally by partitioning database tables across multiple nodes of the processing engine, and is again dependent on the analysis application being used.

Many products now also employ Web-based interfaces allowing visualisation of results via Web browsers. However, the ease of use and utility of this approach is limited by the absence of secure communications protocols and the absence of a session state within Web interaction.

According to Flohr [13], not all applications may be Web-enabled efficiently, such as function-intensive applications for specialised users. Also, it is often difficult to match the visualisation capability available in a standard GUI environment with that available within a browser. Although inefficient at present, Web-based OLAP could provide a cheap approach for delivering sophisticated visualisation for intra- and extra-nets in the future.

This work differs from the above approaches in that the framework is not being targeted at a particular analysis package or algorithm, nor connectivity provided to a particular database — such decisions are left to the analyst, while this work concentrates on developing an infrastructure within which third-party tools may be integrated. For instance, the framework uses Java database connectivity (JDBC) bridges to connect to databases, and consequently, data extraction tasks are delegated to tools capable of processing SQL commands and optimising SQL queries. Similarly, Web interfaces may be easily integrated into the framework using Java applets, with various packages available to support complex user interfaces (such as the SWING library [14]). The framework uses the MPI library to support parallel operations, rather than make direct use of sockets or TCP/IP-based connectivity. The use of MPIs enables scalability, and provides a programmer with a range of abstractions which are not available in a standard programming language. The framework therefore provides a middle ground between using a particular package with a single or a set of analysis routines, and a general-purpose programming language in which the capability to tie together multiple workstations has to be implemented by the programmer.

3. Java-based framework

When comparing data mining approaches for large data sets, the following features are identified as being of particular relevance.

- Deep-memory hierarchy

A deep-memory hierarchy is required to maintain the data physically closer to the processing elements, in cache rather than main memory, for instance. The question of data migration also becomes important as selecting a data distribution across multiple processors becomes significant for improving application performance. The developer should have the option of utilising different cache layers (as are available in most hardware platforms nowadays), and also be able to partition data across the cache, main memory or disk on a single host machine. The framework should provide support for allowing the user to maintain data at various levels in the memory hierarchy, and provide primitives to position data at particular points in the processor pool.

- Load balancing across multiple processors

Load balancing across multiple processors becomes particularly significant when there is a high diversity in the range of available machines. Migration of service needs to be performed transparently, and is aided by the presence of a concurrent process model (threads, for instance) in the underlying language or operating system.

- Computational complexity

Computational complexity of data mining algorithms can be excessive, and being able to implement such algorithms in parallel is of particular benefit when an algorithm can be divided into a set of independent tasks. Neural networks are particularly suitable candidates, due to the explicit parallelism that exists within most neural learning rules based on local updates of processing units, and the neural architecture.

- Quality of results

Quality of results is particularly important for decision support, and the framework should support visualisation to detect anomalies, and enable a means of measurement for calculating the quality of the output generated.

The proposed framework is illustrated in Fig 1, and allows the integration of both commodity and special-

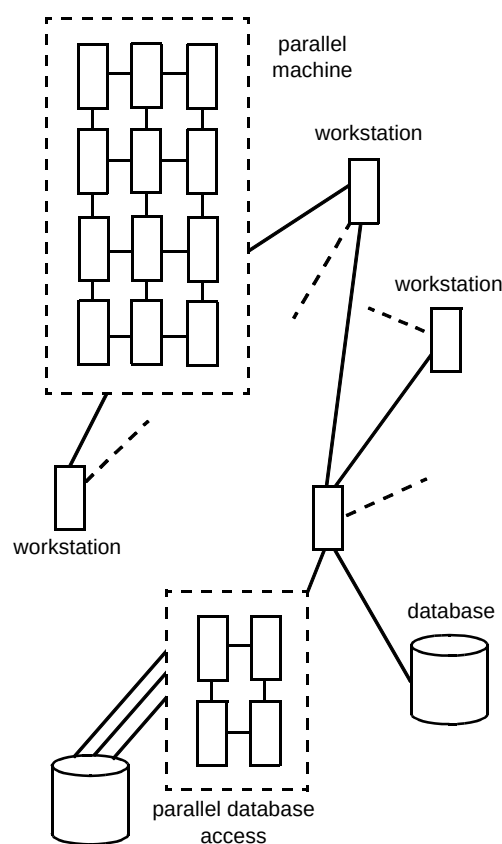


Fig 1 A heterogeneous environment.

purpose hardware resources. To keep the system affordable and deployable, use is made of a network of workstations, which may be linked to multiprocessor systems such as the IBM-SP2 or the Sun E4000, among others. Clusters of such systems may be combined for processing large data sets.

Support for a deep-memory hierarchy and parallelism is provided by distributed data structures, to achieve various data distribution patterns. The presence of a MPIs implies the existence of multiple computation processes, allowing the migration of processes across processor banks for load balancing. The MPI also provides a robust communication mechanism between various Java applications. Hence, a developer may use a communicator process, as in a traditional MPI, and may group processes according to rank to support inter-process communication within and across address spaces. The process topology thus obtained enables a designer to think about the logical process space obtained from the algorithm, rather than the physical set of processors on which the algorithm is run. The virtual processes are subsequently mapped on to the available processors via the MPI parallel library. Hence, a developer does not need to consider the total number of processors available — only the logical process topology has to be developed for the particular analysis approach being considered. This also leads to better scalability, as the underlying processor pool may be altered without affecting the logical process topology.

The framework therefore allows a designer to distribute an application across multiple parallel or single processor machines. Development of a parallel application is simplified by provision of language primitives that

automatically invoke MPI commands via the run-time system. Figure 2 illustrates the overall run-time infrastructure. The framework addresses two significant features in data mining — the placement of data, and the division of computationally intensive queries into a set of independent processes, to be executed in parallel. The underlying infrastructure may be divided into two types of nodes — I/O nodes which are connected via JDBC bridges to data sources and are responsible for retrieving data in parallel, and query nodes which may be pooled together to respond to a query in parallel.

4. Parallel Java

Parallel Java or HPJava [3] implies the combination of MPI libraries for parallel processing with the Java programming language. This enables the use of both thread-based parallelism, and support for process groups using MPI function calls, the motivation being to allow syntax extensions to the base language, such as distributed arrays, available in traditional high performance languages but not supported in Java. The code containing extensions is subsequently translated by a preprocessor to standard Java code and calls to the MPI library. The MPI calls are implemented via a run-time system for performing collective communications, such as broadcasts, and also supports gather/scatter operations for irregular data access, for instance.

HPJava therefore extends the Java programming language to provide support for scientific parallel programming, and combines tools, class libraries and language extensions to support parallel processing paradigms such as shared memory programming, message

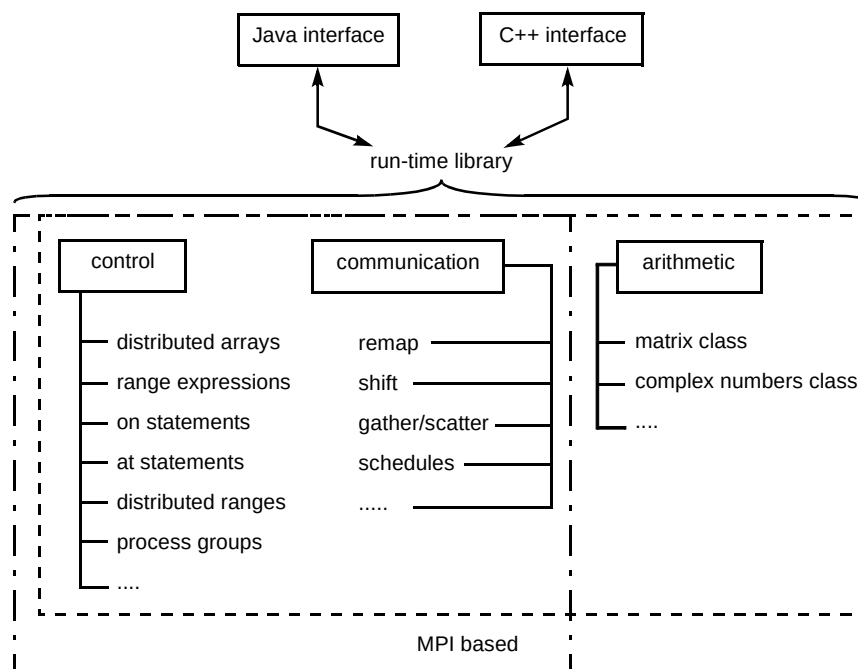


Fig 2 Overall run-time infrastructure.

passing and array-parallel programming. Once such a framework is in place, bindings to higher level libraries and application-specific codes such as CHAOS [15], and ScaLAPACK [16] may also be developed.

One way to extend Java for parallel programming is to introduce characteristic ideas of other high-performance languages, such as the distributed array model and array intrinsic functions and libraries of HPF (high-performance Fortran). The resulting programming model would be single program/multiple data (SPMD), allowing direct calls to the MPI or other communications packages from the HPJava program. Providing distributed arrays as language primitives would allow the programmer to simplify error-prone tasks, such as converting between local and global array subscripts and determining which processor holds a particular element. The compiler for HPJava would make calls to a run-time library and generate underlying Java code. The translator is being implemented in a compiler construction framework developed by members of the Parallel Compiler Run-time Consortium (PCRC) [17].

Within HPJava the PCRC run-time Kernel is referred to as Adlib, and implemented as a C++ library, involving a hybrid of the SPMD and data parallel approaches. The following types of functionalities are provided in Adlib.

- Multidimensional arrays

Multidimensional arrays allow regular section subscripting, similar to Fortran 90 arrays. Such arrays are a language extension and coexist with ordinary Java arrays, for example:

```
float [[,]] a = new float [[5,5]];
int [[,,]] b = new int [[10,n,20]] ;
```

An example of section subscripting is:

```
int [[[ e = a [[2, 2 :]] ;
```

where *e* becomes an alias for the 3rd row of *a*. Adlib, unlike Fortran, does not, however, permit vectors of integers as subscripts. Figure 3 shows how an array can be split across a number of processes, each process acting on a subset of the total array. An array range may also be collapsed, indicating that a complete array may be mapped to a single process, or an array may be identically replicated across a number of processes.

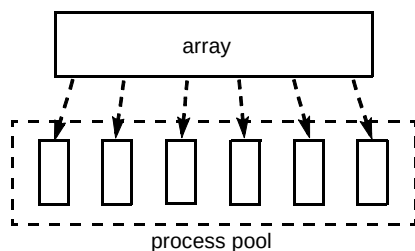


Fig 3 Distributing an array.

- Distributed arrays

Distributed arrays may be viewed as coherent global entities, but their elements are divided across a set of co-operating processes. Distributed arrays are based on the concept of a process array, over which elements of a distributed array are scattered.

For instance, a 2×2 process array can be defined as:

```
Procs2 p = new Procs2(2,2)
```

whereas a 6-element, one-dimensional process array is:

```
Procs1 q = new Procs1(6)
```

- Ranges

A Range object defines a range of integer subscripts, and how such subscripts are to map into a process array dimension. Each value in the range is mapped to the process (or slice of processes) with that co-ordinate. Hence, a distributed range object may appear in place of an integer extent in the constructor of the array, as follows:

```
float [[*,*,]] a = new float
[[x,y,100]] on p;
```

which defines *a* as an *x* by *y* by 100 array of floating point numbers. As the first two dimensions of the array are distributed ranges (the dimensions of *p*), *a* is realised as four segments of 100 elements, one in each of the processes.

- Other extensions

Other extensions include BlockRange which is a subclass of Range and describes a simple block-distributed range of subscripts, Distributed Parallel Loops similar in concept to the FORALL construct of Fortran, and Subranges and Subgroups which can be viewed as a slice of a process array and formed by restricting the process co-ordinates in one or more dimensions to single values. Hence, the active process group or the group over which an array is distributed may be just some slice of a complete process array.

The concepts presented here may be applied to various languages, and are not restricted to Java.

5. A neural network application

A multi-layer perceptron (MLP) neural network using the backpropagation learning rule was developed, as a proof-of-concept application. The neural network topology is varied, and the corresponding effects on performance are measured. According to Nordstrom and Svensson [18], a neural network may be parallelised in six ways.

- Training session parallelism

In this category, different training sessions are started on different processing elements. These sessions may have different starting values for the weights and different learning parameters, such as learning rate, momentum, etc. It is possible to exploit this parallelism if there are no dependencies between the training sessions.

- Training example parallelism

In this category different training examples are assigned to different banks of processing elements. The outputs generated from each neural network executing on a set of processors is combined to generate the final solution, usually by a host machine. According to Nordstrom and Svensson [18], this technique is easy to utilise without a communications overhead and gives an almost linear speed-up with the number of processing elements.

- Layer and forward/backward parallelism

This category involves pipelining computations in the different stages or modes of operation. Hence, multiple operations can be in progress at the same time (corresponding to the 'training' and 'normal' operation mode in the backpropagation algorithm for instance).

- Node (neuron) parallelism

Each node within a neural network computes a function based on the inputs it receives. If the data necessary to achieve this update is local to a node, it is possible for all the nodes to work in parallel.

- Weight (synapse) parallelism

The input to a node is obtained by combining the output from the nodes in the previous layer with a weight value. The weight values are generally maintained at destination nodes. It is possible to perform this operation simultaneously for all inputs to a node if the weight values can be individually combined with outputs from nodes in the previous layer. This is therefore another source of parallelism in a neural network implementation.

- Bit parallelism

This category of parallelism is determined by the preferred implementation style. It involves the use of a data word rather than a serial bit stream. It is relevant for comparison with techniques where a serial bit stream is used in arithmetic calculations.

The use of network, node and data parallelism, as illustrated in Fig 4, was investigated for a three-layer MLP network using the backpropagation learning rule. These

particular parallelisms were chosen as they conform to the data parallel (SPMD) approach followed in HPJava. The other approaches, such as bit parallelism, are more suitable for a hardware implementation, rather than a simulation on a parallel machine, or a cluster of workstations. The number of nodes in the input and hidden layers was changed, and experiments were undertaken to measure the resulting effect on performance.

5.1 Changing the data set size in a neural network

The neural network used has 30 input-layer neurons, 16 hidden-layer neurons and 2 output neurons. The neural network is trained on 'sunspot' activity data obtainable from various Web sites¹.

The number of samples in the training set was varied, and the time to learn measured. Some training samples were repeated to reach a reasonably sized training set. The computer configuration used consisted of four Sun workstations in a cluster, running the Solaris 2.5 operating system. All Sun workstations used JDK 1.1.6 and MPICH software from Argonne National Labs. The results are plotted in Fig 5, and are average times obtained after each processor had been used as the master. Training was repeated by swapping the master processor for each of the four Sun machines. The results indicate that as more samples are added to the data set, the training time does not increase significantly. Communication delays in the HPJava implementation used were significant, and the values provided in the figure have been normalised to the fraction components, to highlight the general trend.

5.2 Changing the number of processors

Changing the number of processors leads to improvements in performance as illustrated in Fig 6. The speed-up is not linear, however, primarily due to the additional costs of communication between processors. The experiment used a logical array of 12 tasks on 2, 3 and 4 Sun workstations, connected on the local network. The Unix 'ping' command was used to calculate the time to send data between the host machine and other workstations for calibration purposes, assuming a data packet of size 64 bytes. Average times are reported in Table 1.

Table 1 Ping times from host to workstations.

Host to workstation time	Time (ms) min/average/max
Host ping	0/0/1
1 workstation	1/1/4
3 workstations	1.67/1.67/4.33

¹ For example — <http://www-isis.ecs.soton.ac.uk/research/nfinfo/fzdata.shtml>

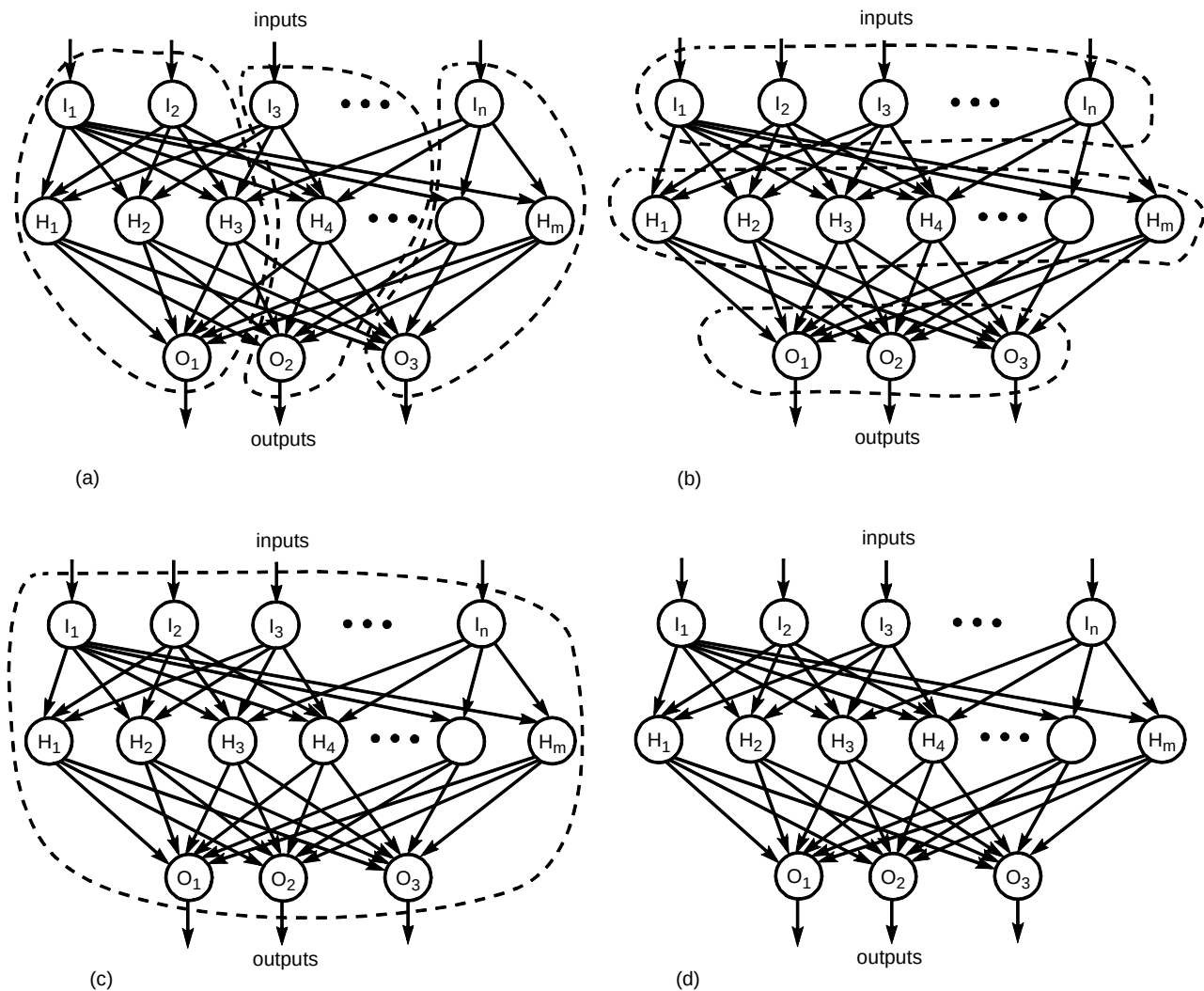


Fig 4 Parallel MLP neural network — (a) vertical parallelism, (b) horizontal parallelism, (c) repetition, (d) data parallelism.

The speed-up obtained when running an algorithm in parallel is the time to execute the best version of the algorithm on a single processor, divided by the total time when executing the same algorithm on several operations in parallel. A linear speed-up results when efficiency increases by 100% for each additional processor added; this is an

ideal scenario which is not generally achievable in a workstation cluster, unless special constraints are placed on the communication mechanism or the size of the datasets. One reason efficiency falls below 100% is because operations running on separate processors need to communicate from time to time. This overhead is absent if

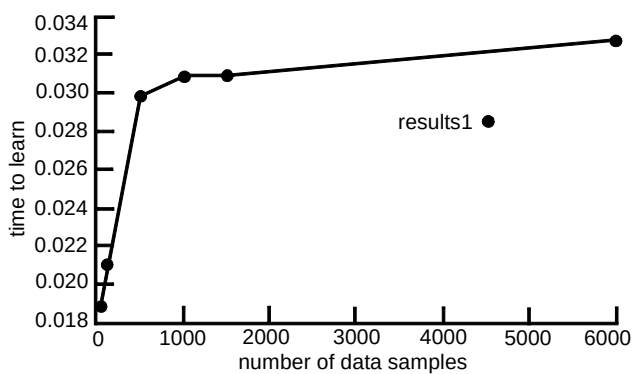


Fig 5 Results — changing the size of the data set.

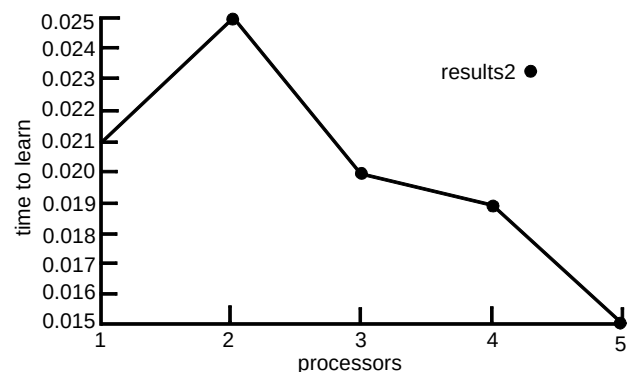


Fig 6 Results — changing the number of processors. The y axis corresponds to the total execution time on a particular processor. The intercept on the y axis corresponds to the execution time on a single machine — in this case the host.

there is only a single processor, and varies with background workload on the machine, and with the amount of network traffic. The aim of the work was to show that parallel processing on commodity hardware using Java (in this case, Sun workstations on an Ethernet) is possible. However, lack of suitably large data sets meant that experiments were performed on a relatively small number of data samples, and for these inter-process communication dominated computation. The experiments successfully demonstrated the execution of the algorithms in parallel on separate machines, and a small but encouraging speed-up was obtained. The speed-up when using four processors is 10%. There is also scalability for data points for up to 500 data samples, as shown in Fig 5.

The conclusions reached are that candidate problems where computation time exceeds communication time are suitable for parallelisation. Also better performance could be obtained if machines used a faster processor, or had lower background workload.

6. Conclusions

A framework for parallel and distributed data mining is proposed. The framework is based on the Java programming language, with support for parallelism provided via MPI libraries. The paper has explained how the framework may be used for distributing a neural network across a cluster of workstations, as a proof-of-concept application. A developer may implement various data mining algorithms on the framework, and may link such algorithms to parallel and sequential data sources via specialised data-gathering nodes. Third-party tools may also be integrated into the framework. The framework is quite general, and may be used in various applications that can be divided into subtasks which are computationally expensive. Further experiments are under way for utilising this framework in a parallel agent environment, using the Aglets [19] mobile agent workbench.

Acknowledgements

We would like to acknowledge the support and suggestions of Steve Corley and Gavin Meggs at BT Laboratories. Part of this work was carried out as part of the BT Research Fellowship scheme, with the Data Mining and Agents groups.

References

- Bradley P, Fayyad U, and Mangasarian O: 'Data Mining: Overview and Optimization Opportunities', in *INFORMS: Journal of Computing* (1998).
- Gropp W, Lusk E and Skjellum A: 'Using MPI', MIT Press (1994).
- Carpenter B, Fox G, Leskiw D, Li X and Wen Y: 'Language Bindings for a Data-Parallel Runtime', NPAC — Syracuse University, Syracuse, New York (1997).
- Albrecht J and Lehner W: 'On-Line Analytical Processing in Distributed Data Warehouses', in *IDEAS Proceedings*, IEEE Computer Society Press (July 1998).
- Quinlan R: 'C4.5: Programs for Machine Learning', Morgan Kaufmann (1997).
- Craven M and Shavlik J: 'Using Neural Networks for Data Mining', *Future Generation Computer Systems* (1997).
- Tabachnick B and Fidell L: 'Using Multivariate Statistics', Addison Wesley (1996).
- Goldberg D E: 'Genetic Algorithms in Search, Optimization and Machine Learning', Addison Wesley (1989).
- Agrawal R, Arning A, Bollinger T, Mehta M, Shafer J and Srikant R: 'The Quest Data Mining System', in *Proc of the 2nd Int Conf on Knowledge Discovery in Databases and Data Mining*, Portland, Oregon (1996).
- Shafer J, Agrawal R and Mehta M: 'SPRINT: A Scalable Parallel Classifier for Data Mining', in *22nd VLDB Proceedings*, Bombay, India (1996).
- Pendse N: 'OLAP Omnipresent', *BYTE* (February 1998).
- Torrent Systems and IBM Corporation: 'White paper: Achieving Scalable Performance for Large SAS Applications', (1997).
- Flohr U: 'OLAP by Web', *BYTE* (September 1997).
- Sun Microsystems: 'Swing and Java Foundation Classes', (1998) — <http://java.sun.com/products/jfc/tsc/swingdoc-static/intro.html>
- Das R, Uysal M, Salz J and Hwang Y-S: 'Communication optimizations for irregular scientific computations on distributed memory architectures', *Journal of Parallel and Distributed Computing*, 22, No 3, pp 462—479 (1994).
- Choi J, Dongarra J J, Ostrouchov S, Petitet A, Walker D W and Whaley R C: 'The design and implementation of the scaLAPACK LU, QR, and Cholesky factorization routines', *Scientific Programming*, 5, pp 173—184 (1996).
- Parallel Compiler Runtime Consortium: 'DARPA project', (1998).
- Nordstrom T and Svensson B: 'Using and Designing Massively Parallel Computers for Artificial Neural Networks', *Journal of Parallel and Distributed Computing*, 14, pp 260—285 (1992).
- Lange D B and Oshima M: 'Programming and Deploying Java Mobile Agents with Aglets', Addison-Wesley (1998).



Omer Rana holds a PhD in Computer Science from Imperial College, London, in parallel architectures and neural algorithms, an MSc in Microelectronics from Southampton University, and a BEng in Information Systems Engineering from Imperial College, London. He is currently a lecturer in the Parallel and Scientific Computation group at Cardiff University, working on Problem Solving Environments and Agent-based High Performance Computing (HPC). He was a visiting research fellow to the North East Parallel Architectures Center in 1998, where he worked under Professor Geoffrey Fox on

the High Performance Java compiler. He was a visiting research fellow to BT Laboratories, working with the data mining and agents group in 1998. He is currently joint co-ordinator for the European JavaGrande forum, an interdisciplinary forum for researchers from academia and industry working to promote the use of Java as the preferred language for HPC. He currently chairs the Java User Group for the South Wales area, involving various industrial and academic collaborators involved in the use and teaching of the Java programming language.



Donald Fisk received a BSc(Hons) in Physics and Astronomy from Glasgow University in 1978, then did some research on General Relativity at Queen Mary and Westfield College, before his first job in virtual machine maintenance/development at Burroughs Machines Ltd in Cumbernauld.

In 1984 he emigrated to Hong Kong where he worked on Compiler Development (Hong Kong Poly), Expert Systems (TI) and Speech Processing (HK Productivity Council), before returning to the UK to work for BT in advanced information processing and

data mining.

In 1995 he completed the BT MSc, for which he developed MORSE, a WWW-based movie recommendation system.